

Software Requirement Engineering

Difference b/w Software Engineering & SRE

SE (Software Engineering)

1. Focuses on designing, developing, testing and maintaining software system.
2. Concerned with entire SDLC.
3. Deals with "how" of software development.
4. Involves programming, coding and software construction.
5. Aims to produce a working software product.

SRE (Software Requirement Engineering)

1. Focuses on defining, analyzing, documenting and maintaining software requirements.
2. Concerned with understanding the needs of stakeholder and users.
3. Involves requirement gathering, analysis, specification and validation.
4. Deals with the "what" of software development.
5. Aims to produce a clear, complete and unambiguous (ضوح) set of requirements.

Classification of requirements:

Functional Requirements:

1. Describe what the system must do.
2. Define the task and functions the system must perform.
3. Examples: login functionality, data processing, user authentication.

Non-functional Requirements:

1. Define the constraints and quality attributes of the system.
2. Describe how the system should perform.
3. Example: Performance, scalability, security, usability.

Business Requirements:

1. Define the business goals & objectives.
2. Describe the needs and constraints of the organization.
3. Examples: Increase sales, reduce costs, improve customer satisfaction.

User Requirements:

1. Describe the needs and expectations of the end-users.

2. Define the tasks and functions

users must perform.

3. E.g. user interface, user experience workflow.

System Requirements:

1. Describe the technical requirements

of the system

2. Define the hardware, software and network infrastructure needed.

3. E.g. operating system, database network protocols.

Domain Requirements:

1. Describe the specific rules & regulations of the domain

2. Define the industry-specific standards and best practices.

3. Examples: healthcare regulations, financial industry standards.

Quality Requirements:

1. Describe the levels of quality and performance expected

2. Define the metrics & benchmarks for evaluation

3. E.g. response time, throughput, error rate.

Requirement Process

It is a systematic approach used in SE to Elicit, analyze, document, validate and manage requirements for a software system.

1) Requirement Gathering:

1. Identify stakeholder and users need
2. Collect requirements through interviews, surveys and observations.
3. Define the scope and boundaries of the project.

Requirement Analysis:

1. Examine and Interpret (know) requirements.
2. Identify conflicts, ambiguities (unclear) and inconsistencies
3. Define the functional and non-functional requirements

Requirements Specification:

1. Document The requirements in a clear and concise manner.
2. Use natural language, diagram and model to describe the requirements.
3. Produce a requirements specification document.

Requirements Verification:

the requirements for completeness,

1. Check the requirements for accuracy and consistency
2. Ensure that requirements are unambiguous and measurable
3. Verify that the requirement meet the stakeholder needs.

Requirement Validation:

1. Ensure that the requirements are correct and meet the user needs
2. Validate requirements through prototyping, testing & feedback.
3. Confirm that requirements are feasible and achievable.

Requirement Management:

1. Track and manage changes to the requirements
2. Ensure that the requirements are up-to-date and relevant
3. Maintain a requirements traceability matrix.

Requirement Communication:

1. Communicate the requirements to Stakeholders and team members
2. Ensure the everyone understands the requirements
3. Use appropriate communication channels and techniques

SRE

Levels/Layers of Requirements

1) Business requirement (Why) (of stakeholders)
 → User software (How) (Expectation)

Functional (What) (Feedback)

BR (User)	UoR (User)	S.R. (Version)
Outline measurable goals & objectives	Reflect specific needs & expectation of users from the system	Identify FRs, NFRs & use case
Define why making project	Describe who & project (How used & which people are used)	Describe how a product is used
Match project goals with stakeholder goals	Describe how users can interact with it	Describe product (software) as functional & non-functional (Babbar Ham) & deliverable
Maintain BRD with requirements, update arrange	create URs and make them them part of BRD	compose SRs & optionally FPS

Requirement Characteristics:

Clear:

- Easy to understand
- Unambiguous (2%)
- Free from technical terminologies

Concise:

- Brief and to the point
- Avoid unnecessary details
- Focus on essential information

Complete:

- Covers all necessary information
- Includes all relevant details
- Leaves no room for misinterpretation

Consistent

- Aligns with other requirements
- Uses consistent terminology
- Avoids contradictions

Feasible

- Realistic and achievable
- Takes into account technical and resource constraints
- Can be implemented within budget and timeline.

Measurable

- Quantifiable and verifiable
(precise way to measure performance)
- include clear metrics and bench marks (reference point to evaluate/improve performance)
- Allows for objective evaluation

Traceable

- Links to business goals and objectives

- Connected to user needs and expectations

- Easy to track and monitor

Verifiable

- Can be tested and validated
- include clear acceptance criteria
- Ensures requirement is met

Analyzing Quality Requirements

Quality requirements also known as non-functional requirements or quality attributes, specify the overall quality characteristics that a system must perform.

1. Performance:

Define how well the system must perform under certain conditions

- Response Time
- Throughput
- Scalability

2. Security:

Address the protection of data, resources and functionality from unauthorized access

- Authentication
- Authorization
- Data protection

3. Usability:

Focus on user experience and ease of use of the system.

- User experience
- Accessibility
- Error handling

4. Maintainability:

Specify how easily the system can be modified, extended or updated over time

- Modularity
- Scalability
- Upgradability

5. Portability:

System can be deployed and run on different hardware, OS & environment.

- Compatibility (ability to operate with other system)
- Adaptability
- Interoperability (communication with 3rd party)

6) Reliability:

specify the system's ability to perform consistently and reliably over time without failure.

- Uptime
- Downtime
- Failure rate